

软件过程改进方案设计文档

课程：软件过程与管理
学院：四川大学软件学院
学期：2026 年春
作者：请填写姓名 / 学号 / 小组成员
提交格式：Markdown
提交渠道：GitHub

二、软件过程改进方案设计文档

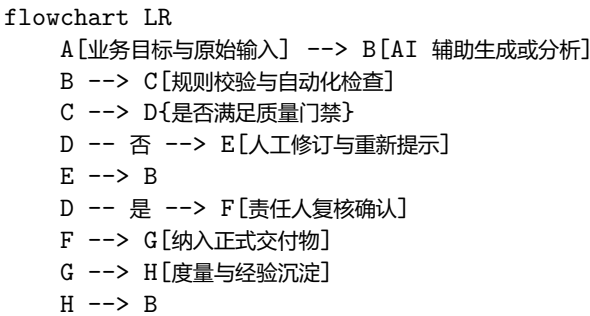
2.1 改进目标

改进后的软件过程目标不是用 AI 替代原有软件工程活动，而是在关键活动中加入可控的 AI 辅助节点，使过程具备更高的效率、覆盖度和反馈速度。具体目标包括：

- 在需求阶段提高用户故事、验收标准和需求冲突识别的完整性。
- 在设计阶段提高架构备选方案和接口契约的充分性。
- 在编码阶段降低样板代码开发成本，并在人工审查前发现低级问题。
- 在测试阶段提高测试用例生成效率、边界条件覆盖率和缺陷发现能力。
- 在运维阶段提高日志分析、异常聚类 and 根因定位效率。
- 建立 AI 输出物的质量标准、复核机制和可追溯记录，避免 AI 幻觉直接进入正式交付。

2.2 总体过程重构思路

改进后的过程采用“AI 生成初稿 + 工程规则校验 + 人工复核确认 + 过程度量反馈”的四步模式。AI 负责扩展候选方案、生成初稿、提示风险和整理信息；工程规则负责检查格式、覆盖率、安全规范和测试结果；人工角色负责业务判断、技术取舍、最终批准和责任确认；度量反馈负责记录 AI 对效率和质量的影响。



2.3 新增角色定义

角色	职责	参与阶段	主要交付物
AI 过程负责人	负责制定 AI 使用规范、工具准入标准、数据安全规则和度量指标	全生命周期	AI 使用规范、工具清单、风险台账
AI 提示词工程师	设计和维护需求、设计、测试、代码审查等场景的提示词模板	需求、设计、测试	提示词模板库、输出样例
AI 输出审核员	对 AI 生成的需求、用例、代码建议和报告进行一致性与可靠性审核	需求、编码、测试	AI 输出审核记录
AI 测试分析师	使用 AI 生成测试用例，评估覆盖率、重复率和可执行性	测试验证	测试用例集、覆盖矩阵、验证报告
数据与安全管理员	负责脱敏数据、权限控制、模型调用日志和敏感信息检查	全生命周期	脱敏记录、安全检查报告

这些角色可以由现有成员兼职承担。例如测试负责人可兼任 AI 测试分析师，项目经理可兼任 AI 过程负责人，架构师可承担部分 AI 输出审核职责。对于课程项目而言，不需要设置独立岗位，但需要在小组分工中明确责任人。

2.4 改进后的关键活动流程

2.4.1 需求分析阶段

flowchart TD

```
A[收集访谈记录与业务背景] --> B[脱敏与输入整理]
B --> C[AI 生成用户故事和验收标准]
C --> D[AI 检查需求冲突与缺失]
D --> E[产品负责人复核]
E --> F{是否确认}
F -- 否 --> G[补充访谈或修订提示词]
G --> C
F -- 是 --> H[形成需求规格说明书]
H --> I[建立需求追踪矩阵]
```

需求阶段的 AI 输出物不能直接作为正式需求。正式需求必须经过产品负责人、客户代表或需求分析师确认。AI 主要承担整理和补全工作，包括将访谈纪要转换为用户故事、为每条需求补充验收标准、识别“同一概念多种表达”“权限边界不清”“异常流程缺失”等问题。

2.4.2 系统设计阶段 系统设计阶段引入 AI 的重点是方案扩展和风险提示，而不是让 AI 单方面决定架构。AI 可以基于需求规格、非功能要求和技术约束生成 2 到 3 个候选设计方案，并列出具优缺点、依赖风险和接口草案。架构师负责评估方案是否满足性能、安全、可维护性和团队技术栈约束。

设计阶段新增交付物包括《AI 设计建议记录》《接口契约 AI 初稿》《架构风险问题清单》。这些交付物作为设计评审附件，不替代正式的概要设计和详细设计文档。

2.4.3 编码实现阶段 编码阶段采用“AI 辅助实现 + 本地自测 + AI 预审 + 人工审查”的方式。

flowchart LR

```
A[开发任务与验收标准] --> B[AI 辅助生成代码或重构建议]
B --> C[开发者修改与本地运行]
C --> D[单元测试与静态检查]
D --> E[AI 代码预审]
E --> F[提交 Pull Request]
F --> G[人工代码审查]
G --> H[合并与构建]
```

编码阶段必须坚持三条规则。第一，AI 生成代码必须由开发者理解后提交，禁止提交无法解释的代码。第二，AI 审查只能作为辅助评论，不能替代人工批准。第三，所有 AI 生成或修改的代码必须通过自动化测试、静态扫描和代码评审。

2.4.4 测试验证阶段 测试阶段是本文的重点改进对象。测试人员将需求规格、接口文档、业务规则、历史缺陷输入 AI 工具，生成测试用例初稿。随后测试人员根据需求追踪矩阵筛选、合并、补充和执行用例，形成最终测试报告。

flowchart TD

```
A[需求规格说明书] --> B[提取测试点]
C[接口文档] --> B
D[历史缺陷] --> B
B --> E[AI 生成测试用例初稿]
E --> F[去重与可执行性检查]
F --> G[测试人员补充边界用例]
G --> H[执行测试]
H --> I[记录缺陷与结果]
I --> J[形成测试报告和覆盖矩阵]
```

2.4.5 运维监控阶段 运维阶段引入 AI 的目标是辅助异常识别和根因分析。系统上线后，监控平台收集日志、指标和链路追踪数据。AI 对异常日志进行聚类摘要，生成可能原因、影响范围和建议处理步骤。运维人员根据实际监控数据确认根因，并将处理经验沉淀到故障知识库。

2.5 工具链选型

改进环节	推荐工具或框架	核心功能	集成方式	预期效率提升指标
需求分析	ChatGPT、Claude、通义千问、Kimi 等 LLM	访谈摘要、用户故事、验收标准、需求冲突检查	在需求评审前生成初稿和问题清单	需求初稿编写时间减少 30% 以上
编码实现	GitHub Copilot、Cursor、Codex 类工具	代码补全、函数生成、重构建议	IDE 插件或代码仓库工作流	常规编码任务耗时减少 20% 到 40%
代码审查	GitHub Copilot Code Review、静态扫描工具	PR 预审、潜在问题提示、建议修改	Pull Request 前置检查	人工审查中低级问题数量减少
测试生成	LLM、Testin XAgent、自动化测试框架	测试点提取、用例生成、脚本草拟	与测试用例库和 CI 流程结合	用例设计时间减少 40% 左右
运维监控	AIOps 平台、日志聚类模型、LLM 摘要	异常检测、日志摘要、根因建议	接入日志平台和告警系统	故障初步分析时间减少 30% 以上

2.6 AI 输出物质量标准

输出物	质量标准	验证方式	责任人
用户故事	角色、目标、价值清晰，验收标准可测试	需求评审、客户确认、追踪矩阵检查	需求分析师
接口草案	字段、类型、状态码、异常响应完整	架构评审、接口 mock、契约测试	架构师
代码建议	可运行、可解释、符合项目规范	单元测试、静态扫描、代码审查	开发者
测试用例	前置条件、步骤、预期结果完整，可执行	去重检查、覆盖矩阵、执行结果	测试负责人
运维摘要	事实与日志一致，结论可追溯	日志抽样核对、复盘会议	运维负责人

2.7 风险评估与应对策略

风险类型	风险描述	影响	应对策略
数据安全	将用户隐私、密钥、业务敏感数据输入外部模型	数据泄露、合规风险	输入前脱敏；禁止上传密钥和真实用户信息；建立工具白名单
模型幻觉	AI 生成不存在的接口、错误规则或不可靠结论	需求错误、代码缺陷、测试误判	所有 AI 输出必须经过人工复核和自动化验证
过度依赖	团队成员不理解 AI 输出，直接复制使用	技术能力下降、质量不可控	要求提交者能解释 AI 输出；保留人工审查责任
版权与许可证	AI 生成代码可能与未知来源相似	法律和合规风险	避免生成大段特定第三方代码；进行许可证检查
偏见与遗漏	AI 根据训练数据偏向常见场景，忽略特殊业务规则	需求覆盖不足	引入领域专家复核和需求追踪矩阵
成本失控	大量调用模型导致费用增加	项目成本上升	设置调用额度、缓存模板、优先本地工具
过程不可追溯	无法说明哪些内容由 AI 生成、谁确认	责任边界不清	记录提示词、版本、输出摘要和审核人

2.8 度量指标设计

指标	定义	计算方式	用途
AI 贡献度	AI 参与生成或分析的交付物比例	AI 辅助交付物数 / 总交付物数	衡量 AI 使用范围
人工采纳率	AI 建议被人工采纳的比例	被采纳建议数 / AI 建议总数	衡量 AI 输出有效性
人工修改率	AI 初稿需要人工修改的比例	修改项数 / AI 输出项数	衡量 AI 输出质量
用例覆盖率	测试用例覆盖需求点的比例	已覆盖需求点 / 总需求点	衡量测试充分性
缺陷前移率	在早期阶段发现缺陷的比例	评审与测试前期发现缺陷 / 总缺陷	衡量质量前移效果
平均修复时间	从缺陷发现到修复完成的平均耗时	总修复时间 / 缺陷数	衡量响应效率

参考文献与资料

1. DORA, *Accelerate State of DevOps Report 2024*, Google Cloud, 2024. <https://dora.dev/research/2024/dora-report/>
2. Stack Overflow, *2024 Developer Survey: AI*, 2024. <https://survey.stackoverflow.co/2024/ai/>
3. GitHub Docs, *Using GitHub Copilot code review*, 2026 年访问. <https://docs.github.com/en/copilot/how-tos/use-copilot-agents/request-a-code-review/use-code-review>
4. Haowei Cheng et al., *Generative AI for Requirements Engineering: A Systematic Literature Review*, arXiv:2409.06741, 2024. <https://arxiv.org/abs/2409.06741>
5. Junwei Liu et al., *Large Language Model-Based Agents for Software Engineering: A Survey*, arXiv:2409.02977, 2024. <https://arxiv.org/abs/2409.02977>
6. 中国信息通信研究院, 《智能化软件工程技术和应用要求 第 3 部分: 智能测试能力》, 2024。
7. GitHub Blog, *Survey reveals AI's impact on the developer experience*, 2024. <https://github.blog/news-insights/research/survey-reveals-ais-impact-on-the-developer-experience/>